ROS3-Limassol: An OpenPELAOS Prototype for Telemetry Liveness Monitoring

Intermittently Connected Robotic Systems

Charalampos Nadiotis

OpenPELAOS, independent open-source project

Initial project idea: Christos Paparistodemou

23 October 2026 | Preliminary technical report

Abstract

Robotic telemetry can stop arriving because of communication disruption, publisher failure, or upstream data interruption. A receiver therefore needs an explicit rule for interpreting the absence of valid messages. This report describes ROS3-Limassol, an OpenPELAOS development prototype comprising a transport-independent Python core, a synthetic unmanned surface vehicle telemetry source, and an optional ROS 2 adapter. The observer evaluates elapsed receiver time since the last accepted message and distinguishes waiting, healthy, and stale states. Schema validation, session identifiers, and increasing sequence numbers prevent several malformed or replayed messages from refreshing that timer. A deterministic experiment with a three-second blackout reproduces stale detection and recovery, accepting 51 of 81 generated samples. Eighteen core tests pass; one ROS graph test is explicitly skipped because ROS 2 is unavailable in the authoring environment. The evidence establishes software behavior under a specified synthetic scenario. Real transport timing, field performance, and research novelty remain unestablished.

Keywords: ROS 2; field robotics; telemetry; liveness monitoring; intermittent connectivity; reproducibility.

1 Introduction

A robotics operator may continue to see an apparently valid position or battery reading after the source has stopped delivering usable updates. The most recent value and the current availability of its source are different observations. ROS3-Limassol begins with the task of making arrival liveness explicit, before attempting navigation, mission execution, or broader autonomy.

The project is developed under OpenPELAOS, with the initial idea credited to Christos Paparistodemou. ROS3-Limassol is the project identifier; version 0.1.0 implements a toolkit that can use ROS 2. The immediate outcome is a small implementation that can be inspected and exercised without a robot or a ROS installation. Its intended ROS mode adds interprocess communication around the same core logic. This is an independent project, rather than an official ROS distribution.

The motivating question is whether a bounded receiver-side observer can report the absence and resumption of accepted telemetry without depending on synchronized sender and receiver clocks. The present contribution is an implementation and deterministic verification baseline [1]. It does not introduce a new failure-detection algorithm or establish an advantage over existing robotics software. Its value at this stage is a precise contract, reproducibility, and an explicit boundary between verified behavior and planned evaluation.

2 Background and technical basis

ROS 2 provides an established modular robotics software architecture. Macenski et al. describe its design and deployment context [2]. Reusing that infrastructure allows the prototype to focus on application-level observations. The adapter follows the Python publisher/subscriber and `ament_python` conventions in the official documentation [3].

ROS 2 quality-of-service settings distinguish history, reliability, and durability, and communicating endpoints require compatible policies [4]. These policies do not, by themselves, define what a particular application should consider a sufficiently recent valid telemetry sample. The prototype therefore evaluates its own acceptance and timeout rules.

Failure-detector theory distinguishes completeness and accuracy, including the possibility that a detector makes mistakes [5]. That distinction helps interpret a timeout as a receiver observation of missing accepted data. It cannot identify which physical component failed, and no formal failure-detector class is claimed here.

Receiver intervals use Python's `time.monotonic()`. Its documented reference point is unspecified, while differences between readings are meaningful and unaffected by system-clock updates [6]. Sender timestamps remain recording metadata. Local arrival liveness is therefore separate from a future model of measurement age or clock synchronization.

3 System model and liveness rule

3.1 Accepted arrivals

Consider one configured robot and a sequence of accepted receiver arrival times $r_0, r_1, \ldots$ on the same monotonic clock. The sequence is non-decreasing. For observation time t, define

$$\mathcal{R}(t) = \{r_i : r_i \le t\}. \tag{1}$$

This set contains arrivals that have passed the application checks. In the default scenario, the last accepted arrival before the blackout is $r_i = 1.9\,\text{s}$. A malformed payload received later does not enter this set.

When $\mathcal{R}(t)$ is nonempty, the most recent accepted arrival has age

$$a(t) = t - \max \mathcal{R}(t). \tag{2}$$

The value is an interval in seconds, rather than the age of the underlying sensor measurement. At $t = 2.9\,\text{s}$, with a last arrival at $1.9\,\text{s}$, Equation (2) gives $a(t) = 1.0\,\text{s}$.

For timeout $\tau > 0$, the observer state is

$$s(t) = \begin{cases} \text{WAITING}, & \mathcal{R}(t) = \emptyset, \\ \text{HEALTHY}, & \mathcal{R}(t) \neq \emptyset \text{ and } a(t) < \tau, \\ \text{STALE}, & \mathcal{R}(t) \neq \emptyset \text{ and } a(t) \ge \tau. \end{cases} \tag{3}$$

With $\tau = 1.0\,\text{s}$, the previous example is stale. A new accepted message at $5.0\,\text{s}$ resets the age to zero and restores the healthy state. Before the first valid arrival, the observer retains the waiting state.

3.2 Observation timing

Suppose no accepted arrival occurs after r_*, and consecutive observer evaluations are separated by at most h seconds. The first stale report satisfies

$$r_* + \tau \le t_{\text{detect}} \le r_* + \tau + h. \tag{4}$$

For $r_* = 1.9\,\text{s}$, $\tau = 1.0\,\text{s}$, and an assumed evaluation-gap bound $h = 0.1\,\text{s}$, the interval is $[2.9, 3.0]\,\text{s}$. The deterministic scenario detects the transition at the lower endpoint.

Equation (4) depends on the evaluation-gap assumption. A nominal ROS timer period does not establish a hard scheduling guarantee on a general operating system. No real-time bound has been measured. Furthermore, a stream of old measurements carrying new valid sequence numbers can satisfy arrival liveness while remaining unsuitable for navigation.

4 Prototype architecture

4.1 Telemetry contract

The core accepts a versioned JSON object containing robot and publisher-session identifiers, sequence number, timestamp, geodetic latitude and longitude, heading, speed, and battery percentage. Units are explicit: degrees, metres per second, percentage, and timestamp nanoseconds. The source produces illustrative signals, rather than a physically consistent vessel dynamics model.

RFC 8259 specifies JSON, excludes non-finite numeric values, and discusses interoperability concerns including duplicate object names [7]. The application contract adds strict field and value checks. All expected fields must be present; extra fields, duplicate keys, and booleans used as numbers are rejected. Numeric ranges and identifier lengths are bounded.

Payloads are limited to 4096 UTF-8 bytes before JSON decoding. The version-1 speed limit is $100\,\mathrm{m\,s^{-1}}$, which is an application bound. These checks bound application parsing and retained state; they do not establish middleware resource bounds.

4.2 Sessions and ordering

Within the active publisher session, accepted sequence numbers must strictly increase. Gaps are permitted. A previously unseen session permits a publisher restart with sequence zero. Known retired sessions are rejected, preventing delayed samples from a known old session from becoming current again.

The monitor retains at most 128 sessions by default. An additional unseen session is rejected once the limit is reached, while the active session may continue. Restarting the monitor clears the history. Persistent replay protection and authenticated session negotiation remain future work.

The model assumes one legitimate publisher for the configured robot identity. It does not authenticate sources or resolve competing publishers. Invalid payloads, unexpected robots, duplicate or out-of-order sequences, retired sessions, and excess new sessions increment a rejection counter without refreshing liveness.

4.3 Transport separation

The validator, simulator, monitor, and local demonstration use only Python's standard library. The default demonstration passes JSON directly between source and observer within one process.

The optional adapter carries that JSON in `std_msgs/msg/String`. Telemetry endpoints use KEEP_LAST, depth 1, BEST_EFFORT, and VOLATILE. Status uses depth 1, RELIABLE, and VOLATILE. These settings follow documented ROS 2 semantics [4]; compatible transport profiles do not establish measurement freshness.

In ROS mode, the observer is configured to evaluate and publish status every $0.1\,\mathrm{s}$ using a steady-clock timer, with separate robot namespaces. A status consumer still needs to detect disappearance of the observer itself. Version 0.1.0 contains no vehicle-command interface, generated ROS message definitions, or MAVLink adapter.

5 Verification method and results

5.1 Deterministic experiment

The scenario uses duration $D = 8.0\,\mathrm{s}$, source rate $f = 10\,\mathrm{Hz}$, and receiver timeout $1.0\,\mathrm{s}$. Sampling includes both endpoints:

$$t_k = \frac{k}{f}, \qquad k = 0, \ldots, \lfloor Df \rfloor. \tag{5}$$

The configured values give indices $0, \ldots, 80$, hence 81 generated samples.

The link model suppresses samples for which $2.0 \leq t_k < 5.0\,\text{s}$. These are indices 20 through 49, so

$$N_{\text{generated}} = 81, \quad N_{\text{suppressed}} = 30, \quad N_{\text{accepted}} = 81 - 30 = 51. \tag{6}$$

Suppression occurs before delivery to the observer, explaining why the base scenario has zero rejected messages despite missing 30 generated samples.

The observed sequence is waiting at $0.0\,\text{s}$ before the first sample, healthy at $0.0\,\text{s}$ after acceptance, stale at $2.9\,\text{s}$, and healthy again at $5.0\,\text{s}$ [1]. The final pre-blackout sequence is 19 and the first resumed sequence is 50.

Detection is $2.9 - 2.0 = 0.9\,\text{s}$ after the configured blackout starts, or $2.9 - 1.9 = 1.0\,\text{s}$ after the last accepted arrival. These intervals use different reference events. Recovery at the $5.0\,\text{s}$ virtual tick demonstrates the state rule rather than a measured network recovery latency.

5.2 Executed software checks

The recorded run discovers 19 test methods: 18 core methods pass and one ROS graph test is skipped because `rclpy` is unavailable. The core checks cover timeout boundaries, recovery, malformed payloads, invalid clocks, sequence ordering, publisher restarts, retired sessions, history bounds, JSON validation, and deterministic command-line output [1]. This is evidence for the exercised behaviors rather than a complete correctness proof or coverage result.

An offline wheel build succeeds. Inspection confirms the core modules, ROS adapter, launch file, package manifest, license, and ament marker. The installed wheel's JSON demonstration also runs from outside the source tree. Python syntax, the XML package manifest, and CI YAML parse successfully.

The ROS package build, launch execution, DDS graph test, and ROS CI job were not executed in the authoring environment. The supplied graph test observes healthy traffic, cancels the publisher timer to model an outage, and resets it to check recovery when run in a suitable ROS environment. The test definition alone does not establish that the transport has passed.

6 Limitations and next evaluation

The deterministic scenario uses virtual time and in-process delivery. It does not exercise discovery delay, scheduling, serialization overhead, queueing, bandwidth constraints, or a physical wireless link. Arrival age and state labels do not certify measurement capture time, position accuracy, or vehicle condition.

The next evaluation should execute the Jazzy build and graph test, then record a real ROS launch with the same blackout. Controlled experiments can vary delay, jitter, outages, and loss while documenting source rates, session handling, and timeouts. Detection and recovery delays require explicitly defined ground-truth events; stale indications during source availability should also be reported. Repeated runs are needed before estimating performance distributions.

Recorded-data replay, a receive-only MAVLink adapter, and companion-computer deployment are proposed extensions. Multi-robot field operation, marine dynamics, new transport performance, and middleware replacement have not been demonstrated. Research novelty requires a broader literature comparison and additional evidence.

7 Conclusion

ROS3-Limassol establishes an OpenPELAOS starting point for explicit telemetry arrival-liveness monitoring. A receiver monotonic clock, strict message acceptance, and bounded session history make its behavior inspectable. The deterministic blackout reproduces stale detection and recovery, and 18 core test methods pass. A successful ROS transport run and a controlled distributed evaluation are the next evidence needed. This report records a preliminary implementation baseline.

Idea attribution

The initial project idea is credited to **Christos Paparistodemou**, following the project initiator's attribution. His public ResearchGate profile is `https://www.researchgate.net/profile/Christos-Paparistodemou`. The link identifies the contributor; the idea credit is supplied by the project initiator. Project development and this report are presented under OpenPELAOS by Charalampos Nadiotis.

Reproducibility and artifact availability

The software targets Python 3.10 or newer; the authoring run used Python 3.12.14. The optional transport target is ROS 2 Jazzy on Ubuntu 24.04. From the project root:

```
python -m ros3_limassol.demo
python -m ros3_limassol.demo -json
python -m unittest discover -s tests -v
```

The accompanying implementation archive includes the Apache-2.0 license, contributor instructions, tests, and validation record. Public repository publication remains pending. The editable XeLaTeX manuscript includes its bibliography and a build guide. This report has not been published to ResearchGate during preparation.

References

[1] C. Nadiotis. *ROS3-Limassol: OpenPELAOS Development Starter*. Version 0.1.0, source package and `docs/VALIDATION.md`, 23 October 2026. Accompanying software artifact, Apache-2.0; no public archival identifier assigned.

[2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, vol. 7, no. 66, article eabm6074, 2022. doi:10.1126/scirobotics.abm6074. Author manuscript: https://arxiv.org/abs/2211.07752.

[3] Open Robotics. Writing a simple publisher and subscriber (Python). *ROS 2 Documentation: Jazzy*. https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html. Accessed 23 October 2026.

[4] Open Robotics. Quality of Service settings. *ROS 2 Documentation: Jazzy*. https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Quality-of-Service-Settings.html. Accessed 23 October 2026.

[5] T. D. Chandra and S. Toueg. Unreliable failure detectors for reliable distributed systems. *Journal of the ACM*, vol. 43, no. 2, pp. 225–267, 1996. doi:10.1145/226643.226647. Author-maintained record: https://www.cs.cornell.edu/info/people/sam/FDpapers.html.

[6] Python Software Foundation. Time access and conversions: `time.monotonic()`. *Python 3.12 Documentation*. https://docs.python.org/3.12/library/time.html#time.monotonic. Accessed 23 October 2026.

[7] T. Bray, Ed. *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259, December 2017. https://www.rfc-editor.org/rfc/rfc8259.html.